

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM
KHOA CÔNG NGHỆ THÔNG TIN

—o—



fit@hcmus

BÁO CÁO ĐỒ ÁN

LAB 2: PAGE TABLE

Giảng viên: ThS. Phạm Tuấn Sơn

Môn học: Hệ Điều Hành

Sinh viên: 23120255 - Lê Tấn Hiệp
23120262 - Tống Dương Thái Hoà
23120264 - Nguyễn Phúc Hoàng

Lớp 23/4

Thành phố Hồ Chí Minh

MỤC LỤC

1. Thông tin sinh viên và mức độ hoàn thiện	3
1.1. Thông tin sinh viên	3
1.2. Mức độ hoàn thiện	3
1.3. Tự đánh giá	3
2. Giới thiệu	4
2.1. Print a page table	4
2.2. Detect which pages have been accessed	4
3. Print a page table	4
3.1. Yêu cầu	4
3.2. Giải pháp triển khai	4
3.3. Ví dụ	5
4. Detect which pages have been accessed	7
4.1. Yêu cầu	7
4.2. Giải pháp triển khai	7
5. Cách dùng file 23120255_23120262_23120264.patch	8
6. Cài đặt test tự động	8
6.1. Hướng dẫn test	8
6.2. Test vmprint	8
6.3. Test print_kpgtbl	9
6.4. Test pgaccess - Test 1: Syscall hoạt động	10
6.5. Test pgaccess - Test 2: Detect một page	11
6.6. Test pgaccess - Test 3: Clear PTE_A	11
6.7. Bảng test cases	13
7. Tài liệu tham khảo	14

1. Thông tin sinh viên và mức độ hoàn thiện

1.1. Thông tin sinh viên

STT	Họ và tên	MSSV	EMAIL
1	Lê Tấn Hiệp	23120255	23120255@student.hcmus.edu.vn
2	Tổng Dương Thái Hoà	23120262	23120262@student.hcmus.edu.vn
3	Nguyễn Phúc Hoàng	23120264	23120264@student.hcmus.edu.vn

1.2. Mức độ hoàn thiện

STT	Yêu cầu	Điểm	Tỉ lệ hoàn thành
1	Print a page table	5	100%
2	Detect which pages have been accessed	5	100%

1.3. Tự đánh giá

- Mức độ hoàn thành: **2/2** yêu cầu (chiếm 100%).
- Số điểm tự đánh giá: **10/10 điểm**.

2. Giới thiệu

Lab 2 - Page Tables tập trung vào việc hiểu sâu và thao tác với page tables trong hệ điều hành xv6. Nhóm em thực hiện 2 nhiệm vụ chính:

2.1. Print a page table

Triển khai hàm `vmprint()` để in cấu trúc page table theo dạng cây phân cấp (tree), giúp:

- Visualization cấu trúc 3-level page table của RISC-V
- Debug và hiểu rõ mapping virtual $\rightarrow$ physical address
- Hỗ trợ học tập và phát triển kernel

2.2. Detect which pages have been accessed

Triển khai syscall `pgaccess()` để detect trang nào đã được access thông qua PTE_A bit, hỗ trợ:

- Garbage collection systems
- Tối ưu hóa quản lý bộ nhớ
- Thuật toán page replacement

3. Print a page table

3.1. Yêu cầu

Implement hàm `vmprint()` để:

- In ra cấu trúc page table theo format hierarchical tree
- Format output:

```
page table 0x[address]
..0: pte 0x[pte_value] pa 0x[pa_value]
.. ..0: pte 0x[pte_value] pa 0x[pa_value]
```

- Tích hợp với `exec()` để in page table của first process (init)

3.2. Giải pháp triển khai

Phương pháp: Sử dụng đệ quy để traverse 3 cấp page table

Luồng hoạt động:

1. Kernel gọi `vmprint(pagetable)` khi init process (PID=1) được exec
2. In địa chỉ base của page table
3. Đệ quy duyệt tất cả 512 PTEs ở mỗi level
4. Với mỗi PTE hợp lệ: in thụt đầu dòng + thông tin PTE

5. Nếu không phải leaf page → đệ quy xuống level tiếp theo
6. Dừng khi đến level 2 hoặc gặp leaf page

Files đã sửa:

- `kernel/vm.c`: Thêm hàm `vmprint()` và `vmprint_helper()`
- `kernel/defs.h`: Khai báo prototype
- `kernel/exec.c`: Gọi `vmprint()` cho init process (PID=1)

3.3. Ví dụ

Kết quả sau khi booting:

```
xv6 kernel is booting
hart 1 starting
hart 2 starting
page table 0x0000000087f52000
..0: pte 0x0000000021fd3801 pa 0x0000000087f4e000
.. ..0: pte 0x0000000021fd3401 pa 0x0000000087f4d000
.. .. ..0: pte 0x0000000021fd3c1b pa 0x0000000087f4f000
.. .. ..1: pte 0x0000000021fd3017 pa 0x0000000087f4c000
.. .. ..2: pte 0x0000000021fd2c07 pa 0x0000000087f4b000
.. .. ..3: pte 0x0000000021fd2817 pa 0x0000000087f4a000
..255: pte 0x0000000021fd4401 pa 0x0000000087f51000
.. ..511: pte 0x0000000021fd4001 pa 0x0000000087f50000
.. .. ..510: pte 0x0000000021fd5807 pa 0x0000000087f56000
.. .. ..511: pte 0x000000002000180b pa 0x0000000080006000
init: starting sh
$
```

Phân tích kết quả:

- `page table 0x0000000087f52000`

Giải thích:

- Đây là **địa chỉ (PA)** của **root page table** mà xv6 đang in ra.
- Nó luôn **căn theo trang 4KB**, nên thường kết thúc bằng `...000` (ở đây `...2000` cũng là bội của `0x1000`).

Ý nghĩa format các dòng `..i: pte ... pa ...`

Ví dụ dòng:

- `..0: pte 0x0000000021fd3801 pa 0x0000000087f4e000`

Giải thích từng phần:

- `..`: biểu diễn **độ sâu** trong cây page table
 - `..` = tầng 1
 - `.. ..` = tầng 2

- ... = tầng 3
- **0:** : **index** trong bảng (mỗi bảng có **512 entry**, index 0..511)
- **pte 0x...** : giá trị PTE (chứa “địa chỉ trang” + “các cờ flags”)
- **pa 0x...** : **địa chỉ vật lý** lấy ra từ PTE
 - nếu PTE là **non-leaf** → **pa** trở tới **page table tầng dưới**
 - nếu PTE là **leaf** → **pa** trở tới **trang dữ liệu/code thật**

Phân tích cây page table

Nhánh 1: Root entry 0 → xuống tầng dưới

- **..0: pte ...3801 pa ...87f4e000** → entry #0 ở root **valid**, trở tới page table con tại **0x87f4e000**
- **...0: pte ...3401 pa ...87f4d000** → entry #0 ở tầng 2 tiếp tục trở xuống page table con tại **0x87f4d000**

Giờ tới tầng 3 (các dòng ...0/1/2/3):

- **...0: pte ...3c1b pa ...87f4f000**
- **...1: pte ...3017 pa ...87f4c000**
- **...2: pte ...2c07 pa ...87f4b000**
- **...3: pte ...2817 pa ...87f4a000**

Đây thường là các **trang thực sự được map** (code/data/stack/special pages) vì đã xuống tới mức leaf.

Chỉ có vài entry (0..3) được in ra → nghĩa là **đa số entry khác invalid**, xv6 không in (đúng yêu cầu “chỉ in PTE valid”).

Giải thích “flags” trong PTE

- V = 1
- R = 2
- W = 4
- X = 8
- U = 16 (các bit khác như A/D... có thể tồn tại nhưng ở đây bạn đang thấy chủ yếu nhóm này)

Nhìn đuôi flags:

- **...3801** và **...3401** → flags = **1** = **V** → **Non-leaf**: chỉ valid, không R/W/X ⇒ trở xuống page table tầng dưới.
- **...3c1b** → flags = **27** = **V + R + X + U** → Trang **user**, **đọc được**, **chạy được** (không ghi) → Thường là **user code/text page**.

- ...3017 và ...2817 → flags = **23** = **V + R + W + U** → Trang **user**, **đọc/ghi**, không execute → Thường là **user data/heap/stack**.
- ...2c07 → flags = **7** = **V + R + W** (không U) → Trang **kernel-only** (user không được vì thiếu U) nhưng vẫn nằm trong pagetable. → Trong xv6, kiểu page “không U nhưng có mặt trong user pagetable” thường là **trang đặc biệt phục vụ kernel** (ví dụ trap-related), vì kernel cần map nó nhưng không muốn user truy cập.
- ...180b → flags = **11** = **V + R + X** (không U) → Trang **kernel-only, read+execute** → Rất giống kiểu **mã chạy khi trap/return** (kernel cần execute, user không được).
- Tóm lại:
 - Có **U** → user được truy cập (code/data user)
 - Không **U** → kernel giữ riêng (trang đặc biệt / kernel helper)

Vì sao có các index “255”, “510”, “511”?

Ý nghĩa:

- Mỗi tầng page table là mảng 512 entry.
- Các index lớn (255, 510, 511) thường xuất hiện khi hệ điều hành map các vùng “gần cuối” của một vùng VA lớn nào đó (tuỳ nhánh).

4. Detect which pages have been accessed

4.1. Yêu cầu

Implement syscall `pgaccess(void *base, int len, void *mask)` để:

- Kiểm tra PTE_A bit của `len` pages bắt đầu từ `base`
- Return bitmask: bit `i` = 1 nếu page `i` đã được accessed
- **Quan trọng:** Clear PTE_A bit sau khi đọc

4.2. Giải pháp triển khai

PTE_A Bit:

- Bit 6 trong PTE, được hardware tự động set khi CPU access page
- Software phải manually clear để detect access tiếp theo

Luồng hoạt động:

1. Parse arguments: `base`, `len`, `mask_addr`
2. Giới hạn `len ≤ 64` (vì bitmask 64-bit)
3. Với mỗi page `i` (0 đến `len-1`):
 - a. Tìm PTE của page

- b. Nếu `PTE_A = 1`: set bit `i` trong mask, rồi clear `PTE_A`
4. Trả mask về user space qua `copyout()`

Files đã sửa:

- `kernel/riscv.h`: Định nghĩa `PTE_A`
- `kernel/sysproc.c`: Hàm `sys_pgaccess()`
- `kernel/syscall.h`, `kernel/syscall.c`: Đăng ký syscall #35
- `user/user.h`, `user/usys.pl`: User-space interface

5. Cách dùng file `23120255_23120262_23120264.patch`

Dùng để lưu trữ và chia sẻ các thay đổi giữa các phiên bản mã nguồn. Có thể sử dụng lệnh `git apply` để áp dụng tệp patch này vào kho lưu trữ, giúp tích hợp các thay đổi mà không cần sử dụng kho lưu trữ trung tâm.

Step 1 — Clone repo của pdos-mit

```
git clone git://g.csail.mit.edu/xv6-labs-2025
cd xv6-labs-2025
git checkout pgtbl
```

Step 2 — Copy file patch vào folder `xv6-labs-2025/` (cùng level với `Makefile`, `README.md`,...)

Step 3 — Apply

```
git apply 23120255_23120262_23120264.patch
```

6. Cài đặt test tự động

6.1. Hướng dẫn test

Bước 1: Build xv6

```
cd xv6-labs-2025
make clean
make qemu
```

Bước 2: Chạy `pgtbltest` trong QEMU

```
$ pgtbltest
```

6.2. Test `vmprint`

Test: Xem output khi boot xv6

Lệnh: Không cần - tự động in khi init process chạy

Output dự kiến:

```
xv6 kernel is booting
hart 1 starting
hart 2 starting
page table 0x0000000087f6b000
..0: pte 0x0000000021fda01b pa 0x0000000087f68000
.. ..0: pte 0x0000000021fd9c17 pa 0x0000000087f67000
...
init: starting sh
$
```

Điều kiện PASS: Page table được in với format đúng (có `..`, `pte`, `pa`)

```
xv6 kernel is booting
hart 1 starting
hart 2 starting
page table 0x0000000087f52000
..0: pte 0x0000000021fd3801 pa 0x0000000087f4e000
.. ..0: pte 0x0000000021fd3401 pa 0x0000000087f4d000
.. .. ..0: pte 0x0000000021fd3c1b pa 0x0000000087f4f000
.. .. ..1: pte 0x0000000021fd3017 pa 0x0000000087f4c000
.. .. ..2: pte 0x0000000021fd2c07 pa 0x0000000087f4b000
.. .. ..3: pte 0x0000000021fd2817 pa 0x0000000087f4a000
..255: pte 0x0000000021fd4401 pa 0x0000000087f51000
.. ..511: pte 0x0000000021fd4001 pa 0x0000000087f50000
.. .. ..510: pte 0x0000000021fd5807 pa 0x0000000087f56000
.. .. ..511: pte 0x000000002000180b pa 0x0000000080006000
init: starting sh
$
```

6.3. Test print_kpgtbl

Test: Gọi syscall kpgtbl() để in page table của process hiện tại

Lệnh: `pgtbltest` (phần `print_kpgtbl`)

Output dự kiến:

```
print_kpgtbl starting
page table 0x...
..0: pte 0x... pa 0x...
```

```
...
print_kpgtbl: OK
```

Điều kiện PASS: In "print_kpgtbl: OK"

```
$ pgtbltest
print_pgtbl starting
va 0x0 pte 0x21FC885B pa 0x87F22000 perm 0x5B
va 0x1000 pte 0x21FC7C5B pa 0x87F1F000 perm 0x5B
va 0x2000 pte 0x21FC781B pa 0x87F1E000 perm 0x1B
va 0x3000 pte 0x21FC7417 pa 0x87F1D000 perm 0x17
va 0x4000 pte 0x21FC7007 pa 0x87F1C000 perm 0x7
va 0x5000 pte 0x21FC6CD7 pa 0x87F1B000 perm 0xD7
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFB000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFC000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFD000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFE000 pte 0x21FD08C7 pa 0x87F42000 perm 0xC7
va 0x3FFFFFFF000 pte 0x2000184B pa 0x80006000 perm 0x4B
print_pgtbl: OK
print_kpgtbl starting
page table 0x0000000087f25000
..0: pte 0x0000000021fc8401 pa 0x0000000087f21000
.. ..0: pte 0x0000000021fc8001 pa 0x0000000087f20000
.. .. ..0: pte 0x0000000021fc885b pa 0x0000000087f22000
.. .. ..1: pte 0x0000000021fc7c5b pa 0x0000000087f1f000
.. .. ..2: pte 0x0000000021fc781b pa 0x0000000087f1e000
.. .. ..3: pte 0x0000000021fc7417 pa 0x0000000087f1d000
.. .. ..4: pte 0x0000000021fc7007 pa 0x0000000087f1c000
.. .. ..5: pte 0x0000000021fc6cd7 pa 0x0000000087f1b000
..255: pte 0x0000000021fc9001 pa 0x0000000087f24000
.. ..511: pte 0x0000000021fc8c01 pa 0x0000000087f23000
.. .. ..510: pte 0x0000000021fd08c7 pa 0x0000000087f42000
.. .. ..511: pte 0x000000002000184b pa 0x0000000080006000
print_kpgtbl: OK
```

6.4. Test pgaccess - Test 1: Syscall hoạt động

Mục đích: Kiểm tra syscall pgaccess() không crash

Cách hoạt động:

1. Allocate 3 pages mới
2. Gọi pgaccess() để check access bits

Output dự kiến:

```
pgaccess_test: Test 1 - Kiem tra syscall co hoat dong
-> Syscall hoat dong, abits = 0x0
pgaccess_test: Test 1 - OK
```

Điều kiện PASS: Syscall return thành công, không crash

```
pgaccess_test: Test 1 - Kiem tra syscall co hoat dong
-> Syscall hoat dong, abits = 0x0
pgaccess_test: Test 1 - OK
```

6.5. Test pgaccess - Test 2: Detect một page

Mục đích: Kiểm tra pgaccess detect đúng page được access

Cách hoạt động:

1. Allocate 5 pages
2. Clear access bits (gọi pgaccess lần 1)
3. Access CHỈ page 2: `buf[2 * PGSIZE] = 'X'`
4. Gọi pgaccess() lần 2 để check

Output dự kiến:

```
pgaccess_test: Test 2 - Detect mot page duoc access
-> abits = 0x4 (mong doi bit 2 = 1)
-> Bit 2 da duoc set dung!
pgaccess_test: Test 2 - OK
```

Điều kiện PASS: `abits & 0x4 != 0` (bit 2 được set)

```
pgaccess_test: Test 2 - Detect mot page duoc access
-> abits = 0x4 (mong doi bit 2 = 1)
-> Bit 2 da duoc set dung!
pgaccess_test: Test 2 - OK
```

6.6. Test pgaccess - Test 3: Clear PTE_A

Mục đích: Kiểm tra kernel CLEAR bit PTE_A sau khi đọc

Tại sao quan trọng?

- Nếu không clear, lần gọi tiếp sẽ thấy bit = 1 mãi
- Không thể detect access MỚI trong tương lai

Cách hoạt động:

SETUP: Allocate 3 pages, clear existing bits

LẦN 1: Access page 1

- Gọi pgaccess()
- Mong đợi: abits = 0x2 (bit 1 set)

LẦN 2: KHÔNG access gì mới

- Gọi pgaccess()
- Mong đợi: abits = 0x0 (đã clear!)

LẦN 3: Access page 2

- Gọi pgaccess()
- Mong đợi: abits = 0x4 (bit 2 set)

Output dự kiến:

```
pgaccess_test: Test 3 - Kiểm tra clear PTE_A (QUAN TRỌNG!)
Setup: Đã clear access bits
Lần 1: Access page 1...
-> abits = 0x2
-> PASS: Bit 1 đã set
Lần 2: Không access, kiểm tra clear...
-> abits = 0x0 (mong đợi = 0)
-> PASS: Đã clear, abits = 0
Lần 3: Access page 2 mới...
-> abits = 0x4
-> PASS: Bit 2 đã set
pgaccess_test: Test 3 - OK (Clear PTE_A hoạt động đúng!)
```

Điều kiện PASS:

- Lần 1: `abits & 0x2 != 0`
- Lần 2: `abits == 0` ← Chứng minh clearing hoạt động
- Lần 3: `abits & 0x4 != 0`

```

pgaccess_test: Test 3 - Kiểm tra clear PTE_A (QUAN TRỌNG!)
Setup: Đã clear access bits
Lần 1: Access page 1...
-> abits = 0x2
-> PASS: Bit 1 đã set
Lần 2: Không access, kiểm tra clear...
-> abits = 0x0 (mong đợi = 0)
-> PASS: Đã clear, abits = 0
Lần 3: Access page 2 mới...
-> abits = 0x4
-> PASS: Bit 2 đã set
pgaccess_test: Test 3 - OK (Clear PTE_A hoạt động đúng!)

```

6.7. Bảng test cases

STT	Test	Mục đích	Điều kiện PASS	Kết quả
1	print_pgtbl	Đọc PTE qua syscall	In được PTEs	OK
2	ugetpid_test	Kiểm tra ugetpid	PID khớp	OK
3	print_kpgtbl	vmprint hoạt động	In page table dạng cây	OK
4	pgaccess Test 1	Syscall hoạt động	Không crash	OK
5	pgaccess Test 2	Detect 1 page	bit 2 = 1	OK
6	pgaccess Test 3	Clear PTE_A	Lần 2: abits = 0	OK
7	superpg_fork	Super pages fork	Không crash	OK
8	superpg_free	Super pages free	Không crash	OK

7. Tài liệu tham khảo

[1] R. Cox, F. Kaashoek, and R. Morris, "xv6: a simple, Unix-like teaching operating system," Chapter 3: Page Tables [Online]. Available:

pdos.csail.mit.edu/6.828/2023/xv6/book-riscv-rev3.pdf. Accessed: 27-Dec-2025.

[2] RISC-V Foundation, "The RISC-V Instruction Set Manual, Volume II: Privileged Architecture," Version 20211203 [Online]. Available:

github.com/riscv/riscv-isa-manual. Accessed: 27-Dec-2025.

[3] Massachusetts Institute of Technology, "Lab: Page Tables," 6.1810/2023 - Operating System Engineering [Online]. Available:

pdos.csail.mit.edu/6.1810/2023/labs/pgtbl.html. Accessed: 27-Dec-2025.